

Firewall Configuration using iptables

Objective

To understand the basics of Linux firewall management using **iptables** by viewing existing firewall rules, creating rules to allow and block specific network traffic, verifying the rules, and finally removing them.

Tools Used

- Kali Linux
- iptables
- Terminal

Procedure

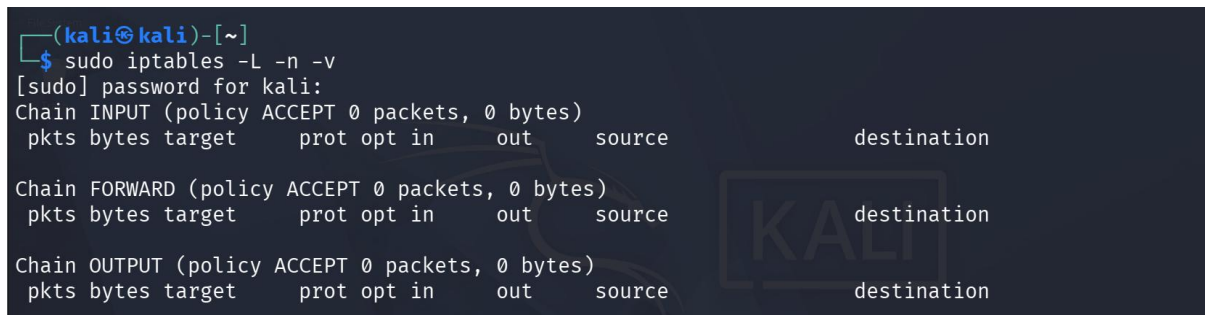
Step 1: Display Existing Firewall Rules

The following command was used to display the current firewall rules:

sudo iptables -L -n -v

Initially, no custom firewall rules were configured. All three default chains (**INPUT**, **FORWARD**, and **OUTPUT**) followed the **ACCEPT** policy.

Screenshot



```
(kali㉿kali)-[~]
$ sudo iptables -L -n -v
[sudo] password for kali:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination
```

Step 2: Allow SSH Traffic

A rule was added to allow incoming SSH connections on port **22**.

sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

Step 3: Block Telnet Traffic

A rule was added to block incoming Telnet connections on port **23**.

sudo iptables -A INPUT -p tcp --dport 23 -j DROP

REPORT BY MANASVI RAJPUT

Step 4: Verify the Firewall Rules

The configured rules were verified using:

sudo iptables -L -n -v

The output showed:

- TCP traffic on **port 22** is **ACCEPTED**.
- TCP traffic on **port 23** is **DROPPED**.

Screenshot

```
(kali㉿kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

(kali㉿kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 23 -j DROP

(kali㉿kali)-[~]
$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 4 packets, 312 bytes)
 pkts bytes target    prot opt in     out     source                 destination            tcp dpt:22
    0    0 ACCEPT    tcp  --  *      *      0.0.0.0/0              0.0.0.0/0              tcp dpt:23
    0    0 DROP     tcp  --  *      *      0.0.0.0/0              0.0.0.0/0              tcp dpt:23

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                 destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                 destination
```

Step 5: Remove the Firewall Rules

After demonstrating the firewall configuration, all custom rules were removed using:

sudo iptables -F

The firewall table was checked again:

sudo iptables -L -n -v

The output confirmed that the custom rules had been successfully removed and only the default ACCEPT policies remained.

Screenshot

```
(kali㉿kali)-[~]
$ sudo iptables -F

(kali㉿kali)-[~]
$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 11 packets, 802 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination
```

Observation

- Initially, no user-defined firewall rules were present.
- An **ACCEPT** rule was successfully created for SSH traffic (TCP Port 22).
- A **DROP** rule was successfully created for Telnet traffic (TCP Port 23).
- The configured rules were visible in the INPUT chain when listed.
- After flushing the firewall, all custom rules were removed successfully.

Conclusion

The experiment successfully demonstrated the basic functionality of **iptables** in Linux. Firewall rules were created to allow SSH traffic and block Telnet traffic, verified using the firewall rule list, and finally removed using the flush command. This exercise provided practical experience in configuring and managing firewall rules to control network traffic in a Linux environment.